

Rank candidates the way a **great recruiter** would.

Not by matching keywords — by understanding who actually fits the role, rejecting fabricated profiles, and weighing who's genuinely hireable.

28%

OF WHO WE RECOMMEND IS INVISIBLE TO
KEYWORD SEARCH

0

FABRICATED PROFILES IN OUR TOP-100

100K

CANDIDATES RANKED, CPU-ONLY, IN
BUDGET

THE PROBLEM

"Recruiters go through hundreds of profiles and still miss the right person. Not because the talent isn't there — but because **keyword filters can't see what actually matters.**"

A keyword filter drops the engineer who wrote *"built the recommendation engine serving the homepage"* for a JD that asks for *"ranking systems"* — same work, different words. **That person is the hire you just lost.**

One engine. Three things keyword tools can't do.

01

Reasons beyond keywords

Hybrid retrieval (embeddings + lexical) over the JD's *own* requirements + a cross-encoder rerank — matches meaning, not just words.

02

Rejects fakes

A role-independent consistency auditor vetoes internally-impossible profiles by their contradictions — 8 yrs at a 3-yr-old company, expert skill with 0 months.

03

Weighs hireability

Schema-driven behavioral signals down-weight stale, unresponsive candidates — adapting to whatever signal fields a dataset has.

JD-agnostic by construction: validated on **2,484 real resumes across 21 categories** — AI, nurse, accountant, chef, aviation — with **no per-category code**.

We read the JD like a recruiter — not a search box.

Any free-text or structured JD is parsed into a **weighted requirement model**, per skill:

"Senior AI Engineer... embeddings, retrieval, ranking, NDCG, Python. 5–9 yrs."

↓ parsed into

MUST-HAVE

embeddings

retrieval

ranking models

hybrid search

NDCG

Python

SENIORITY BAND

5–9 years

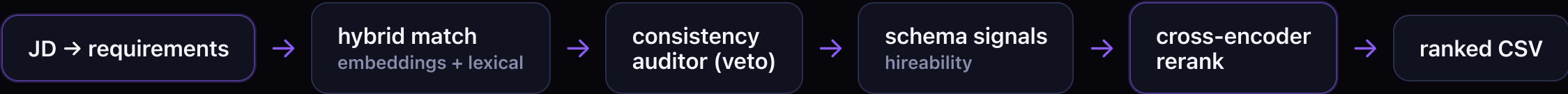
DISQUALIFIERS

research without production

keyword-only AI

Relevance is computed against **these** requirements — so an off-role candidate scores ~0 for any role, automatically.

Two-stage retrieval, then an honest score.



The scoring law is a single, JD-agnostic **relevance gate**: $final = R^{1.1} \cdot (0.45 + 0.55 \cdot Q) \cdot veto$ — where **R** = relevance to *this* JD's requirements (dominant) and **Q** = generic quality (seniority, trust, behavioral). Generic strength can never carry an irrelevant candidate.

METRIC (LABELED MULTI-JD EVAL)	SPINE	HYBRID
mean composite	0.95	0.97
honeypot rate in top-10 (trap-heavy suite)	0%	0%
zero-keyword paraphrase fits in top-10	10/10	10/10

THE RESULT THAT MATTERS

28/100

of the people we shortlist are
invisible to keyword search.

On the real Redrob 100K, a keyword filter ranks 28 of our top-100 candidates **outside its own top 100** — a recruiter on keyword search would never see them. (60 fall below keyword-rank #50.)

RESCUED BY MEANING — THEIR OWN WORDS

"Senior engineer who has spent the last several years building systems that connect users with..." we rank #14 · keyword filter #102 · Senior AI Engineer

"My main learning has come from shipping real systems... recommendation and ranking..." we rank #18 · keyword filter #103 · Senior Data Scientist

We catch the profiles that lie.

A role-independent consistency auditor flags **internal contradictions** — not keywords — and vetoes impossible profiles out of the shortlist entirely.

🚩 VETOED — FABRICATED TENURE

A profile claims a skill for 53 months but states only 1.3 years total experience. The two facts can't both be true → demoted out of the top-100.

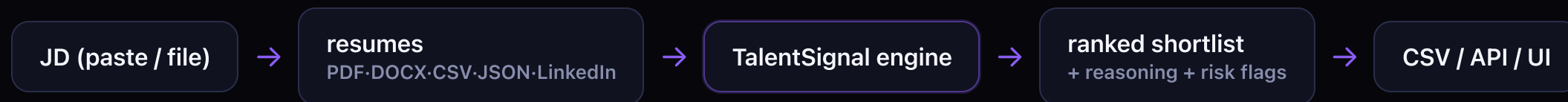


INTERNALLY-IMPOSSIBLE PROFILES IN OUR SUBMITTED TOP-100

Found by contradiction, not keyword. Every kept candidate's reasoning quotes their own sentence — audited, zero hallucination.

This is the brief's "how do you handle suspicious profiles" question — answered with a worked example, not a claim.

Any JD, any candidates, any format → trusted shortlist.



RECRUITER APP

Studio UI — paste a JD, drop resumes, live explainable shortlist with honest fit labels.

INTEGRATE

REST API + Python SDK — drop into any portal/ATS.

AGENTIC

MCP server — 7 tools any AI agent can call.

One engine, four surfaces, zero per-category code.

JD ingest → weighted requirement model

Hybrid match (embeddings + lexical)

Consistency auditor (honeypot veto)

Schema-driven signals (any vocabulary)

Relevance-gate scoring → cross-encoder rerank

Grounded reasoning + risk report

Reproducible & in budget: rank-time is CPU-only, no-network, <5 min / <16 GB. Embeddings precomputed offline.

- Spine engine: zero dependencies, always a valid CSV
- Hybrid engine: precomputed numpy index, numpy-only at rank time
- One rank() facade → UI, REST, SDK, MCP
- 158 tests · official validator passes · deterministic

Validated, in budget, reproducible.

100,000

CANDIDATES RANKED

~30s

RANK TIME, CPU-ONLY,
OFFLINE

PASS

ORGANIZERS' OFFICIAL
VALIDATOR

0

HONEYPOTS IN TOP-100

0.97

HYBRID MEAN COMPOSITE
(0.95 SPINE)

10/10

ZERO-KEYWORD PARAPHRASE
FITS

158

TESTS PASSING

21

REAL CATEGORIES VALIDATED

Every number sourced in `outputs/eval/METRICS.md` — no contradictions, each labeled by engine.

Build something real. Make hiring smarter.

- **GitHub repo** — clean, working code, 158 tests
- **Ranked CSV** — top-100, passes the official validator
- **This deck** — approach, methodology, results
- **Live demo** — Studio UI ranks the real 100K

 **TalentSignal** — the right people for any role.

28%

THE PEOPLE GREAT RECRUITERS KEEP AND KEYWORD FILTERS LOSE
— AND WE SURFACE THEM, WITH PROOF.